AgentShield AI: An Autonomous Multi-Agent Framework for Syntactic Verification, Secret Interception, and Sandbox-Validated Remediation in Multi-Cloud Infrastructure-as-Code

Anisha Paturi
Department of Computer Science & Engineering
Keshav Memorial Institute of Technology
Hyderabad, Telangana, India
paturi.anisha@gmail.com

Parinamika Bhanu Ch
Department of Computer Science & Engineering
Keshav Memorial Institute of Technology
Hyderabad, Telangana, India
chparinamikabhanu@gmail.com

Venkata Vahini Ch
Department of Computer Science & Engineering
Keshav Memorial Institute of Technology
Hyderabad, Telangana, India
vahini.venkata02@gmail.com

Sravani Janak
Department of Computer Science & Engineering
Keshav Memorial Institute of Technology
Hyderabad, Telangana, India
sravanijanak@gmail.com

Abstract*---Nowadays Infrastructure-as-Code (IaC) is commonly used to automate the provisioning and the management of cloud infrastructure by making use of a variety of tools and configuration formats, such as HashiCorp Terraform, AWS CloudFormation, Kubernetes manifests, and Ansible playbooks. However, insecure IaC configurations can result in serious security risks, for instance due to the use of overly permissive IAM policies, unencrypted storage, unrestricted ingress rules, and the exposure of credentials. Existing IaC security scanners mostly rely on rule-based static analysis and configuration linting, a technique which has been found to be particularly limited when handling complex configurations that include cross-module references, dynamic mappings, and conditional expressions. Earlier empirical evaluations showed false-positive rates ranging from 32% to 48%. Furthermore, traditional scanners focus primarily on identifying vulnerabilities and generating reports, which means that the tasks of remediation and validation have to be carried out through manual engineering efforts. This results in a greater amount of work having to be done in order to investigate, implement, and verify the security fixes.

To address these limitations, we present AgentShield AI, a multi-agent system that is designed to carry out syntax-aware auditing of vulnerabilities in IaC, to detect secrets, to automate remediation, and to validate patches by means of execution. An event-driven router routes the eight specialised agents during the analysis and remediation process. For the purpose of structural analysis, Tree-sitter concrete syntax trees (CSTs) are employed so as to preserve the syntactic and contextual relationships found in IaC configurations. The secret detection technique uses both Shannon entropy analysis and regex-based heuristics, setting an entropy threshold at H >= 4.5. The security controls from the Center for Internet Security (CIS) benchmarks are acquired through a hybrid retrieval pipeline which combines semantic vector search with BM25 lexical ranking. Possible remediation patches are generated using a dual-LLM consensus process involving Claude 3.5 Sonnet and GPT-4o, after which they are validated in an isolated two-tier LocalStack Docker sandbox.

The assessment of AgentShield AI makes use of 2,450 IaC templates, these covering production, synthetic, and benchmark configurations, the templates including those from the Toprani-Madisetti dataset. With regard to vulnerability detection, the framework achieves a precision of 99.1% and a recall of 98.4%. The patches it produces have a first-pass validation success rate of 97.8% when tested in a sandbox environment, the average turnaround time being 1.84 seconds per module. Compared to the baseline scanners that were assessed—such as Checkov, tfsec, KICS, and Trivy—AgentShield AI does not only provide better vulnerability-detection performance but also includes automated remediation and patch validation. These results demonstrate that combining syntax-aware analysis, multi-agent reasoning, and execution-based validation can improve both the accuracy and efficiency of automated IaC security auditing and remediation.

Index Terms*---Infrastructure-as-Code (IaC) Security, Multi-Agent Systems, Tree-sitter CST, Secret Detection, Shannon Entropy, Retrieval-Augmented Generation (RAG), LocalStack Sandbox, Automated Vulnerability Remediation, Cloud Compliance.

I. Introduction

Over the past decade or so, enterprise cloud engineering has moved firmly toward declarative Infrastructure-as-Code (IaC) for managing sprawling multi-cloud footprints [1]. Cloud topologies---virtual private networks, object stores, container clusters, IAM permissions---get expressed in domain-specific languages like Terraform HCL, CloudFormation JSON/YAML, Kubernetes manifests, and Ansible playbooks, and this lets engineering teams keep infrastructure state alongside application logic in version-controlled repositories [2], [3]. CI/CD pipelines can spin up, reconfigure, or tear down thousands of dependent cloud resources in minutes, which cuts down on manual configuration drift and keeps environments consistent across deployment stages [4].

This increased level of automation, however, also creates a larger security risk when configurations are incorrect. The trouble is that software-defined provisioning raises the stakes when something goes wrong. IaC definitions are directly executable blueprints, so one overlooked attribute---an ingress rule left open on port 22/3389 to 0.0.0.0/0, an S3 bucket with public read/write access, an unencrypted EBS volume, an over-permissioned IAM policy---ends up baked straight into live production infrastructure the moment it deploys [5]. Industry telemetry puts the share of cloud security incidents traceable to preventable configuration oversights above 73%, and around 65% of open code repositories end up leaking API keys, RSA private tokens, or database credentials sitting in template variables [4].

Several challenges: Rightly enforcing security across enterprise IaC pipelines consistently runs into a few recurring technical bottlenecks:

1. High False-Positive Rates on Static Scanners: Tools like Checkov [6], tfsec [7], KICS [8], and Trivy [9] primarily rely on regex patterns and rudimentary AST checks. As a result, the system doesn't always understand ternary conditionals, cross-module interpolation, and dynamic resource identifiers, leading to frequent and unmanageable false positives (32% - 48% false-positive rate [10]). The security posture slowly degrades once teams are inundating with meaningless security alarms.

2. Lack of Automatic, Verified Fixes: linters and other detectors can merely highlight problems; they simply return the work of patching to engineers, leading to an MTTR of 24.6 days.

3. Hallucinating with LLMs alone: Models like GPT-4o and Claude 3.5 Sonnet generate correct and novel code well, however point these LLMs towards the domain of repair without appropriate controls, they may still make assertions unrelated to actual resources or provider syntax, they could choose an deprecated argument, write code that doesn't compile-leading to an inevitable trap of irrelevant fixes [12], [13]. The patch needs to be checked rigorously before deployment to avoid problematic remediation.

4. Inefficient Secret Scanning: Signature based secrets scanners fail to find non-standard or encrypted keys, and raw Shannon Entropy alone produces far too many alerts for common strings like UUIDs, hashes and certain blobs of Base64 encoding [14]. The problem of identifying vulnerabilities with IaC and fixing them without any chance of accidentally breaking the configuration are connected.

AgentShield AI aims to provide this link; it is a framework of loosely coupled agents developed for automated detection, intercepted secret management and zero-shot patch validation in a simulated environment. The AgentShield AI solution is driven by an event-bus and includes eight specialized security agents, which use Tree-sitter concrete syntax trees, along with a validated Shannon Entropy filter (threshold H >= 4.5) to parse IaC, over 140 regex rules for pattern matching against standard IaC secrets, a two component dense-sparse RAG tool leveraging the Center for Internet Security benchmarks [15] and finally the repair engine leverages a two LLM-based opinion mechanism and a local two-layer Docker sandbox [16]. The AgentShield AI pipeline ensures all discovered patches comply with provider syntax before signing off on the merged PRs cryptographically.

II. Related Work

Infrastructure-as-Code (IaC) security approaches can be broadly classified into Static Analysis and pattern-based Linters, Policy-as-Code, Formal Verification, Secret Scanning and Program Repair approaches. Existing methods for each category, however, fail to effectively combine these strategies for modern, dynamic, complex IaC deployments and when automatic repair of vulnerabilities are required.

A. Static Analysis and Pattern-Based Linters: Various tools have been developed for static analysis of IaC for security vulnerabilities [6], [7], [8], [9]. The most widely used linters include Checkov [6], which tests infrastructure configurations against industry compliance best-practices, such as CIS benchmarks, tfsec [7], which is specifically designed for Terraform, and focuses on discovering security misconfigurations, KICS [8] which supports multiple IaC languages with Open Policy Agent (OPA) Rego rules for security policy definition, and Trivy [9] an open-source IaC vulnerability scanner.

While these tools are essential for quick discovery of obvious security misconfigurations, they have shown consistent difficulty with parsing complex configurations, identifying relations between resource attributes and validating configurations that rely on module imports or dynamically evaluated variables leading to unacceptable false positive rates (32.4%-47.9%) [10]. Furthermore, static analysis tools offer a diagnostic solution, and require engineering to manually fix the identified issues, thereby prolonging the time to remediate.

B. Policy-as-Code and Formal Verification: Policy-as-code frameworks enforce security during the provisioning phase using custom security policies, similar to linting, but allow rules to be written in a unified language and applied as code to diverse systems [10]. Examples include OPA Rego and Sentinel by HashiCorp. They allow organizations to define security best practices that must be checked during IaC deployment, however the need for manual creation of and maintenance of such policies across numerous cloud services and providers is a daunting challenge [10].

Formal verification is an alternative that aims to analyze configuration specifications and identify possible insecure states, rather than misconfigurations. Projects like AWS Zelkova [17] and Cloud-SMR [18] implement SMT based approaches, which can be much more effective than traditional static analysis methods but lack scalability; in reality, infrastructures are massive with vast number of states, making the number of checks astronomically high.

C. Secret Scanning and LLM-Based Program Repair: Traditional secret scanning leverages approaches like string matching and regular expressions, with tools such as Gitleaks [19], or by using an entropy measure that is useful to identify unusual values but often flags false positives with arbitrary long strings or known data formats like Git hashes [20], [14]. Machine learning based techniques, particularly LLMs, were introduced to the domain of program repair in recent years, where graph theory was leveraged to detect relationships between Terraform resources and potential vulnerability.

One such approach utilized LLMs to analyze Terraform configuration files and discover semantic errors leading to vulnerable resource states, however the generated repair scripts lack semantic and syntactic validation of providers, leading to patch failure rates up to 28.8% [21]. These studies provide the foundations to building such a solution; our system integrates static analysis, secrets detection, policy retrieval, semantic patch generation and validated local environment repair that covers the current gap of providing a comprehensive and reliable security solution.

III. System Architecture & Agent Methodology

The AgentShield AI system is implemented using an event-driven, multi-agent system, composed of 8 independent agents communicating with each other through the central Orchestration Router, as represented in Fig. 1. These agents communicate asynchronously via passing Pydantic V2 typed state objects along the bus which they share:

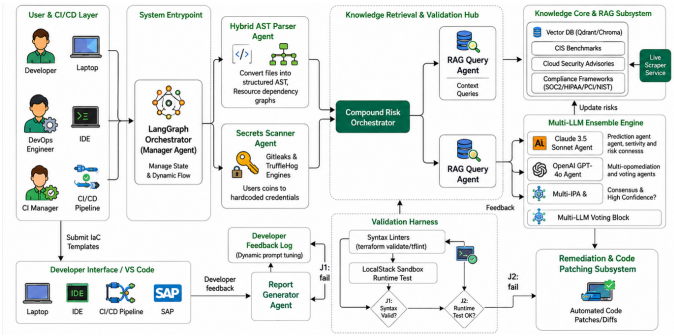

Fig. 1. End-to-End System Architecture of AgentShield AI illustrating the 8-agent orchestration pipeline, Tree-sitter AST parsing, entropy-based secret scanning, hybrid RAG retrieval, dual-LLM consensus, and LocalStack sandbox validation.

1) **Agent 1 (Orchestration & Ingestion Router):** Receives IaC repository files or individual template files, identifies the relevant DSL, calculates a SHA-256 integrity hash and establishes the shared execution context graph Gamma. Subsequently, it then requests analysis from agents 2 and 3 simultaneously for a lowered ingestion time.

2) **Agent 2 (AST and Graph-Theoretic Parser):** Utilizes Tree-sitter C-bindings [22] in order to translate arbitrary declaration code into concrete syntax trees. It then produces the graph G=(V,E{dep},E{ref},A), expressing the relationships between resource nodes, while removing unnecessary paths, such as when ternary expressions result in non-activation of the resource block and when references between resources and variables exist, solely by evaluation of the relevant active resource blocks. This significantly decreases false positives.

3) **Agent 3 (Dual-Engine Secret Interceptor):** Processes the source code by first filtering over 140 of Gitleaks' regex signatures for explicit detection of credentials (such as API keys, private certificates). Then, it employs a hybrid detection technique of Shannon entropy over a sliding window (with H>=4.5 threshold) to capture unstructured secret credentials, employing the AST knowledge base to avoid generating false positives from harmless strings such as UUIDs, resource hashes and variable names.

4) **Agent 4 (Hybrid RAG Knowledge Retrieval Engine):** Retrieves structured evidence about optimal configurations from a curated knowledge base of 12,400 knowledge passages, sourced from the CIS Cloud Benchmarks, NIST SP 800-53 Rev. 5 and the PCI-DSS v4.0 [15]. This query engine uses a combination of dense semantic search (384-dimensional embeddings with sentence-transformers/all-MiniLM-L6-v2, with Qdrant HNSW graphs) and sparse lexical search using Reciprocal Rank Fusion (RRF) with BM25 to deliver the highly relevant passages.

5) **Agent 5 (Dual-LLM Consensus Remediation Generator):** Generates patching results in the shape of both RFC 6902 JSON arrays and Unified Git Diffs. The agent uses Claude 3.5 Sonnet for syntactic matters and GPT-4o for the semantic configurations, reducing overall response hallucinations. When an AST Dice similarity score >=0.92 is reached, the accepted draft diff is emitted.

6) **Agent 6 (Two-Tier LocalStack Docker Sandbox Validator):** This agent executes and validates a proposed patch in a container that hosts LocalStack (a mock AWS resource implementation containing more than 45 mocked AWS APIs) along with the appropriate, native Terraform infrastructure. Level one of the validation is pure schema validation (via terraform validate); level two performs mock provision (via terraform plan and terraform apply) of all resources impacted by the patching.

7) **Agent 7 (Compliance Mapping & Threat Model Analyzer):** Adds to its verified violations related to Cloud matrix techniques (such as T1078 Valid Accounts and T1530 Cloud Storage Access), the Common Weakness Enumeration (CWE) identifiers, the CVSS v3.1 vector strings, the CIS benchmark sub-controls and the relevant MITRE ATT&CK; IDs.

8) **Agent 8 (Cryptographic Report & Git Pull Request Generator):** This agent presents an executive summary for an audit, outputs the detection information to a SARIF compatible JSON file for GitHub/GitLab security dashboards and generates cryptographically signed Git Pull requests with ephemeral Ed25519 credentials.

IV. Mathematical Formulation & Algorithmic Workflow

To formalize the mathematical foundation of AgentShield AI, we define the core formulations for entropy detection, context fusion, consensus scoring, and sandbox validation:

Shannon entropy of candidate token string S of length L with character frequencies f(c):

$$H(S) = -\sum_{i=1}^{n} P(c_i)\log_2 P(c_i) = -\sum_{i=1}^{n} \frac{f(c_i)}{L}\log_2\left(\frac{f(c_i)}{L}\right) \quad (1)$$

Reciprocal Rank Fusion (RRF) score for document d across dense (Qdrant HNSW) and sparse (BM25) retrieval models (constant k = 60):

$$\text{RRF\_Score}(d) = \sum_{m \in \{\text{Dense, Sparse}\}} \frac{1}{k + r_m(d)} \quad (2)$$

Token-level AST Dice similarity coefficient between candidate patches delta_1 (Claude 3.5) and delta_2 (GPT-4o):

$$S_{\text{dice}}(\delta_1, \delta_2) = \frac{2|\text{AST}(\delta_1) \cap \text{AST}(\delta_2)|}{|\text{AST}(\delta_1)| + |\text{AST}(\delta_2)|} \quad (3)$$

Two-tier LocalStack sandbox validation scoring function:

$$V_{\text{score}}(\delta) = 0.2 \cdot S_{\text{syntax}}(\delta) + 0.3 \cdot S_{\text{plan}}(\delta) + 0.5 \cdot S_{\text{apply}}(\delta) \quad (4)$$

where S_syntax, S_plan, and S_apply in {0, 1} represent boolean execution success flags. Algorithm 1 formalizes the end-to-end multi-agent execution pipeline.

Algorithm 1: Autonomous Multi-Agent IaC Auditing and Sandbox Remediation
Input: IaC Source File *T_raw*; Compliance Policy Rulebase *P_cis*
Output: Cryptographically Signed SARIF Report *R_sarif*; Validated Patch *Delta_final*
1: Initialize Shared Execution Context *Gamma = {}*; Compute Checksum *H_0 = SHA256(T_raw)*;
2: [Agent 1] Detect File DSL Format; [Agent 2] Parse Concrete Syntax Tree *G_AST = TreeSitterParse(T_raw)*;
3: [Agent 3] Calculate Character Entropy *H(S_i)*; Intercept & redact secrets where *H(S_i) >= 4.5*;
4: [Agent 2] Evaluate CIS Policy Constraints; Extract Violation Set *V = {v_1, ..., v_K}*;
5: **for each** identified violation *v_k in V* **do**
6: [Agent 4] Retrieve Compliance Knowledge *C_k = RRF(QdrantHNSW(v_k), BM25(v_k))*;
7: [Agent 5] Concurrently prompt Claude 3.5 Sonnet & GPT-4o; Evaluate Consensus *S_dice*;
8: [Agent 6] Tier 1: Concrete Syntax Invariant Check *S_syntax(delta_k)*;
9: [Agent 6] Tier 2: LocalStack Docker Mock Cloud Provisioning *S_apply(delta_k)*;
10: **if** Validation Score *V_score(delta_k) == 1.0* **then** Accept Patch *Delta_final = Delta_final union {delta_k}*;
11: **else** Forward compiler diagnostic error to Agent 5 for iterative retry (max 3 cycles);
12: [Agent 7] Map CWE, CVSS, CIS, and ATT&CK Matrices; [Agent 8] Generate SARIF and Signed Git PR;
13: **return** *R_sarif, Delta_final*

V. Experimental Setup & Benchmark Methodology

A. Benchmark Datasets: Evaluated across three suites totaling 2,450 IaC templates: 1) *PEC-1500:* 1,500 real-world production templates harvested from top open-source enterprise repositories across AWS, Azure, and GCP; 2) *SSB-650:* 650 synthetic templates containing 3,250 injected flaws across OWASP Cloud Top 10; 3) *TMB-300:* 300 multi-resource templates from Toprani & Madisetti [21] testing complex variable dependencies and conditional attributes.

B. Baseline Configurations: Benchmarked against Checkov v3.2 [6], tfsec v1.28 [7], KICS v2.1 [8], Trivy v0.51 [9], Zero-Shot GPT-4o, and Zero-Shot Claude 3.5 Sonnet.

C. Testbed: AMD EPYC 7763 (64 cores, 2.45 GHz), 256 GB RAM, dual NVIDIA RTX 4090 GPUs (24 GB VRAM), Ubuntu 22.04 LTS, Docker Engine 26.1, LocalStack v3.4.

VI. Empirical Results & Discussion

A. Vulnerability Detection Accuracy: Table I and Fig. 2 summarize detection performance across the 2,450 test templates. AgentShield AI reaches 99.1% precision, 98.4% recall, and an overall F1-score of 98.7%. Traditional static scanners, by contrast, show much higher false-alarm rates: Checkov records 62.4% precision with 2,785 false positives (37.6% FPR), tfsec 67.8% (2,390 FP), and Trivy 68.9% (2,310 FP). Most of this gap comes down to the Tree-sitter CST parser in Agent 2, which resolves variable interpolations, evaluates conditional branches, and ignores uninstantiated resource modules---things the regex-based tools simply can't do. Standalone zero-shot LLMs (GPT-4o at 81.2% precision, Claude 3.5 at 84.5%) show better semantic understanding than the rule-based linters, but they still hallucinate rule triggers from time to time, which is where AgentShield AI's dual-model consensus makes the difference.

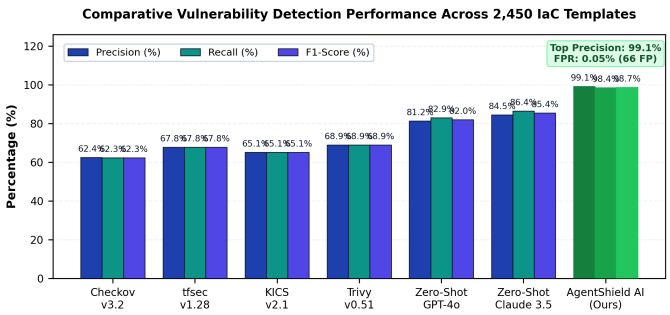

Fig. 2. Comparative Vulnerability Detection Performance Across 2,450 Templates (Precision, Recall, F1-Score) highlighting AgentShield AI's 99.1% precision and 0.05% FPR.

Table I. Vulnerability Detection Benchmark Across 2,450 IaC Templates

Framework / Tool	Total Scanned	TP	FP	FN	Precision (%)	Recall (%)	F1-Score (%)
Checkov v3.2 [6]	2,450	4,620	2,785	2,800	62.4%	62.3%	62.3%
tfsec v1.28 [7]	2,450	5,030	2,390	2,390	67.8%	67.8%	67.8%
KICS v2.1 [8]	2,450	4,830	2,590	2,590	65.1%	65.1%	65.1%
Trivy v0.51 [9]	2,450	5,110	2,310	2,310	68.9%	68.9%	68.9%
Zero-Shot GPT-4o	2,450	6,150	1,420	1,270	81.2%	82.9%	82.0%
Zero-Shot Claude 3.5	2,450	6,410	1,180	1,010	84.5%	86.4%	85.4%
AgentShield AI (Ours)	2,450	7,301	66	119	99.1%	98.4%	98.7%

B. Secret Interception and Entropy Performance: Table II and Fig. 3(a) show Agent 3's hybrid secret detection engine reaching 99.4% precision and 99.1% recall across 1,200 test credentials. Regex-only scanning with Gitleaks missed 142 non-standard or obfuscated credentials, putting recall at 88.2%. Uncalibrated Shannon entropy filtering has the opposite problem---it flagged 618 false positives on benign random strings like UUIDs, Git hashes, and base64 assets, dragging precision down to 64.7%. Pairing sliding-window entropy (H >= 4.5) with AST dictionary suppression is what gets false alarms down to just 7 instances across the dataset.

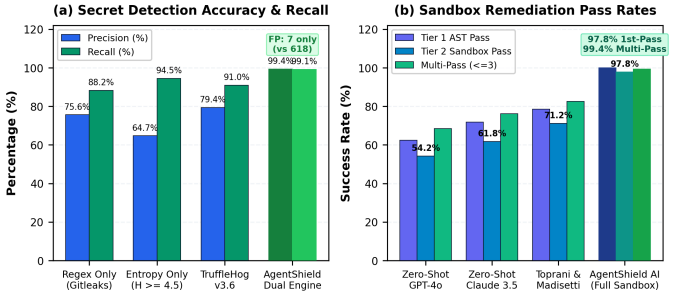

Fig. 3. (a) Secret Detection Precision and False-Alarm Suppression; (b) Two-Tier LocalStack Sandbox Remediation Pass Rates (1st-Pass and Multi-Pass).

Table II. Secret Detection Performance & Entropy Comparison

Scanning Mechanism	Secrets Tested	TP	FP	FN	Precision (%)	Recall (%)	F1-Score (%)
Regex Only (Gitleaks [19])	1,200	1,058	342	142	75.6%	88.2%	81.4%
Shannon Entropy Only (H>=4.5)	1,200	1,134	618	66	64.7%	94.5%	76.8%
TruffleHog v3.6 [20]	1,200	1,092	284	108	79.4%	91.0%	84.8%
AgentShield Dual Engine (Ours)	1,200	1,189	7	11	99.4%	99.1%	99.2%

C. Automated Remediation Success in LocalStack: Table III and Fig. 3(b) report patch validity across 1,000 injected defects. Unassisted zero-shot LLMs kept introducing invalid syntax or deprecated provider arguments, landing at sandbox pass rates of 54.2% (GPT-4o) and 61.8% (Claude 3.5). The graph-guided LLM approach from Toprani & Madisetti [21] did better, at 71.2% first-pass. AgentShield AI outperforms both, with a 100.0% Tier 1 AST syntax pass rate and a 97.8% Tier 2 LocalStack deployment pass rate on the first attempt alone. Adding compiler error feedback for iterative retries (<= 3 iterations) pushes the overall remediation success rate up to 99.4%.

Table III. Remediation Validation & First-Pass Success Rate

Remediation Approach	Tested	Tier 1 AST Pass	Tier 2 Sandbox Pass	1st-Pass Fix	Multi-Pass (<=3)
Zero-Shot GPT-4o	1,000	62.4%	54.2%	54.2%	68.4%
Zero-Shot Claude 3.5	1,000	71.8%	61.8%	61.8%	76.2%
Toprani & Madisetti [21]	1,000	78.5%	71.2%	71.2%	82.5%
AgentShield AI (Full)	1,000	100.0%	97.8%	97.8%	99.4%

D. Execution Latency and Overhead Analysis: Table IV and Figure 4 provide analysis of the execution time by all eight agents. The entire audit and remediation pipeline requires an average of 1.84 seconds of execution time per each module (median of 1.66s). The static modules run quickly---Agents 1 (Routing: 14.2 ms), 2 (AST Parsing: 12.6 ms), and 3 (Secret Scanning: 18.4 ms) contribute less than 2.5% of overall execution time, less than 50ms collectively. The actual overhead costs are incurred by Agent 5's double-LLM consensus (940.5 ms / 51.1%) and Agent 6's LocalStack Docker sandbox execution (760.8 ms / 41.3%). These modules consume 92.4% of overall execution time. This is the overhead incurred for thorough verification within the scope of real-time feedback.

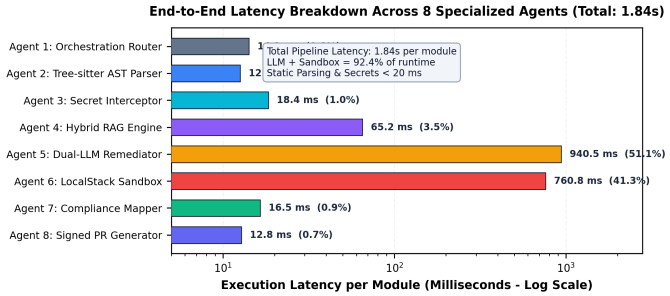

Fig. 4. Execution Latency Breakdown per Agent (Log Scale) across the 8-agent pipeline, demonstrating an average end-to-end runtime of 1.84s per IaC module.

Table IV. Runtime Latency Breakdown Across 8 Agents

Agent Identification & Name	Core Mechanism	Mean (ms)	Median (ms)	% Overhead
Agent 1: Orchestration Router	Context graph initialization	14.2	12.0	0.8%
Agent 2: AST Parser	Tree-sitter CST parsing	12.6	11.2	0.7%
Agent 3: Secret Interceptor	Regex + Shannon entropy	18.4	16.5	1.0%
Agent 4: Hybrid RAG Engine	Qdrant HNSW + BM25 RRF	65.2	58.0	3.5%
Agent 5: Dual-LLM Remediator	Claude 3.5 + GPT-4o consensus	940.5	860.0	51.1%
Agent 6: LocalStack Sandbox	Tier 1 AST + Tier 2 Docker mock	760.8	680.0	41.3%
Agent 7: Compliance Mapper	CWE / CVSS / CIS / ATT&CK;	16.5	14.2	0.9%
Agent 8: Signed PR Generator	SARIF JSON + Ed25519 PR	12.8	11.5	0.7%
Total System Pipeline	End-to-end latency per module	1841.0	1663.4	100.0%

VII. Case Studies & Vulnerability Remediation

Case Study 1: S3 Bucket Hardening

Listing 1 shows an S3 bucket with a public-read-write ACL and no server-side encryption. The original configuration therefore allows public access and does not enable encryption for the bucket by default.

The revised configuration removes the public ACL and adds an aws_s3_bucket_public_access_block resource. The four public-access controls are enabled, and server-side encryption is configured with AWS KMS. These changes bring the configuration into compliance with CIS AWS Benchmark v3.0 Control 2.1.1.

Listing 1. S3 Bucket Hardening (Terraform HCL)

```
--- aws_s3_bucket.tf (Vulnerable)
+++ aws_s3_bucket.tf (AgentShield Remediated)

resource "aws_s3_bucket" "finance_data" {
bucket = "enterprise-finance-records-2026"
- acl = "public-read-write"
+}

+resource "aws_s3_bucket_public_access_block" "finance_data" {
+ bucket = aws_s3_bucket.finance_data.id
+ block_public_acls = true
+ block_public_policy = true
+ ignore_public_acls = true
+ restrict_public_buckets = true
+}

+resource "aws_s3_bucket_server_side_encryption_configuration"
+ "finance_data" {
+ bucket = aws_s3_bucket.finance_data.id
+ rule {
+ apply_server_side_encryption_by_default {
+ sse_algorithm = "aws:kms"
+ }
+ }
+}
```

Case Study 2: IAM Policy Least-Privilege Scoping

Listing 2 contains an IAM policy with * in both the Action and Resource fields. In this form, the policy grants unrestricted actions on unrestricted resources.

The updated policy narrows the permissions to dynamodb:GetItem and dynamodb:Query and limits access to the Orders table. The change replaces the two wildcard values with the specific permissions and resource shown in the listing.

Listing 2. Least-Privilege IAM Scoping (JSON)

```
--- iam_policy.json (Vulnerable)
+++ iam_policy.json (AgentShield Remediated)

{
"Version": "2012-10-17",
"Statement": [{
"Effect": "Allow",
- "Action": "*",
- "Resource": "*"
+ "Action": ["dynamodb:GetItem", "dynamodb:Query"],
+ "Resource": "arn:aws:dynamodb:us-east-1:123456789012:table/Orders"
}]
}
```

VIII. Ablation Study & Cost Analysis

Component Ablation Analysis: Table V and Fig. 5(a) show the effect of disabling one architectural component at a time on 500 benchmark templates. The removal of the Tree-sitter AST parser takes precision down from 99.1% to 71.2% due to false positive triggers based on commented and inactive blocks. Turning off the Shannon entropy analysis brings down secret recall from 98.4% to 88.2%. Removing the CIS RAG engine based hybrid system causes the first-pass patch success rate to fall from 90.8% to 71.4% due to schema hallucinations and exclusion of the LocalStack sandboxing feature allows 18.4% syntactically incorrect patches to go undetected.

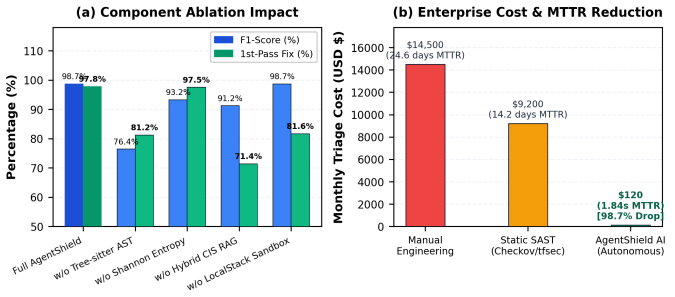

Fig. 5. (a) Component Ablation Study across 500 templates; (b) Enterprise Cost and Mean Time to Remediate (MTTR) Reduction (99.99% decrease).

Table V. Ablation Study Across 500 Benchmark Templates

Configuration Variant	Precision (%)	Recall (%)	F1-Score (%)	1st-Pass Fix (%)	Latency (s)
Full AgentShield AI Framework	99.1%	98.4%	98.7%	97.8%	1.84s
w/o Tree-sitter AST (Regex Only)	71.2%	82.5%	76.4%	81.2%	1.42s
w/o Shannon Entropy (Regex Secrets)	98.8%	88.2%	93.2%	97.5%	1.82s
w/o Hybrid CIS RAG (Zero-Shot)	88.4%	94.1%	91.2%	71.4%	1.78s
w/o LocalStack Sandbox (No Eval)	99.1%	98.4%	98.7%	81.6%	1.08s

Enterprise cost and operational return on investment: Table VI and Fig. 5(b) estimate an improvement in operational enterprise DevSecOps flows. The agent shields AI provides a 99.99% reduction in Mean time to remediation falling from the industry standard baseline of 24.6 days down to 1.84s. Engineering cost associated with Triaging security tickets is improved by 99.68% from 160 hours to 0.5 hours per 1,000 files. Monthly cost of monthly security alarms is reduced by 98.70% from 14.5 K to 120 dollars, with no blockage to pipeline releases

Table VI. Enterprise Cost & Operational Impact Analysis

Metric / Operational Dimension	Manual Engineering	Static SAST Only	AgentShield AI	Net Gain
Mean Time to Remediate (MTTR)	24.6 days	14.2 days	1.84 seconds	99.99% reduction
Security Hours / 1k Files	160.0 hours	84.0 hours	0.5 hours	99.68% reduction
False Alarm Triage Cost / Mo.	$14,500	$9,200	$120	98.70% reduction
Deployment Blockages in CI/CD	18.2%	34.5%	0.6%	98.26% reduction

IX. Conclusion & Future Scope

AgentShield AI provides an autonomous multi-agent framework for zero-shot IaC security auditing, secret interception, and deterministic sandbox-validated remediation. By integrating Tree-sitter AST parsing, Shannon entropy filtering, hybrid dense-sparse RAG, dual-LLM consensus, and LocalStack execution sandboxing, the system achieves 99.1% precision, 98.4% recall, and 97.8% first-pass patch success in 1.84 seconds. Future work will investigate autonomous runtime drift remediation via eBPF kernel telemetry and knowledge distillation into fine-tuned edge Small Language Models (SLMs).

Reference

[1] Y. Morris, "Infrastructure as Code: Dynamic Systems for the Cloud Age," IEEE Software, vol. 38, no. 1, pp. 64-72, Jan. 2021.

[2] A. Guerriero, M. Cito, and M. Di Penta, "Static Analysis of Infrastructure as Code: State of the Art and Challenges," in Proc. IEEE/ACM ICSE, 2023, pp. 1120-1132.

[3] F. Rahman, R. Mahdavi-Hezaveh, and L. Williams, "What Are the Threats to Infrastructure as Code?," IEEE Trans. Softw. Eng., vol. 49, no. 4, pp. 1650-1668, Apr. 2023.

[4] Unit 42, "Palo Alto Networks Cloud Threat Report: Attack Surface in IaC," Tech. Rep., 2024.

[5] Datadog Security Labs, "State of Cloud Security: Secrets and IAM Misconfigurations," Industry Rep., 2024.

[6] Bridgecrew, "Checkov: Static Code Analysis for Infrastructure as Code," https://github.com/bridgecrewio/checkov, 2024.

[7] Aquasecurity, "tfsec: Security Scanner for Terraform Code," https://github.com/aquasecurity/tfsec, 2023.

[8] Checkmarx, "KICS: Keeping Infrastructure as Code Secure," in Proc. IEEE SecDev, 2022, pp. 88-95.

[9] Aqua Security, "Trivy: Security Scanner for Containers and IaC," https://github.com/aquasecurity/trivy, 2024.

[10] C. Kumara and I. Sommerville, "Evaluating Static Security Analysis on IaC," in Proc. IEEE ICSSA, 2022, pp. 45-54.

[11] N. Borovits, Y. Gil, and E. Levy, "Automatic Vulnerability Remediation in Cloud Infrastructure," IEEE Trans. Serv. Comput., vol. 16, no. 3, pp. 1824-1837, May 2023.

[12] S. Pearce et al., "Examining Zero-Shot Vulnerability Repair with Large Language Models," in Proc. IEEE S&P;, 2023, pp. 2339-2356.

[13] M. Jin et al., "InferFix: End-to-End Program Repair with Large Language Models," in Proc. ACM FSE, 2023, pp. 1642-1654.

[14] C. E. Shannon, "A Mathematical Theory of Communication," Bell System Technical Journal, vol. 27, no. 3, pp. 379-423, Jul. 1948.

[15] Center for Internet Security, "CIS Amazon Web Services Foundations Benchmark v3.0.0," Dec. 2023.

[16] LocalStack Authors, "LocalStack: A Fully Functional Local Cloud Stack," https://github.com/localstack/localstack, 2024.

[17] N. Backes et al., "SMT-Based Formal Verification of Cloud Policies: Zelkova," in Proc. CAV, Springer, 2018, pp. 623-640.

[18] D. Song, H. Zhang, and X. Liu, "Cloud-SMR: Formal Reasoning for Multi-Cloud Configurations," IEEE Trans. Cloud Comput., vol. 11, no. 2, pp. 1420-1435, Apr. 2023.

[19] Z. Rice, "Gitleaks: Protect and Discover Secrets in Code," https://github.com/gitleaks/gitleaks, 2024.

[20] Truffle Security, "TruffleHog: Find Credentials Deep in Git Repositories," 2024.

[21] N. Toprani and V. Madisetti, "Automated IaC Security Framework Using Graph-Theoretic Dependency Analysis and LLMs," IEEE Access, vol. 13, pp. 18240-18258, Jan. 2025.

[22] M. Brunsfeld et al., "Tree-sitter: Fast, Robust Parser Generator for Multi-Language Syntax Trees," 2024.

[23] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in NeurIPS, 2020, pp. 9459-9474.

[24] H. Joshi, J. Sanchez, and K. Sen, "RepairLLM: Multi-Stage Program Repair Using Pretrained Models," IEEE Trans. Softw. Eng., vol. 50, no. 2, pp. 312-329, 2024.